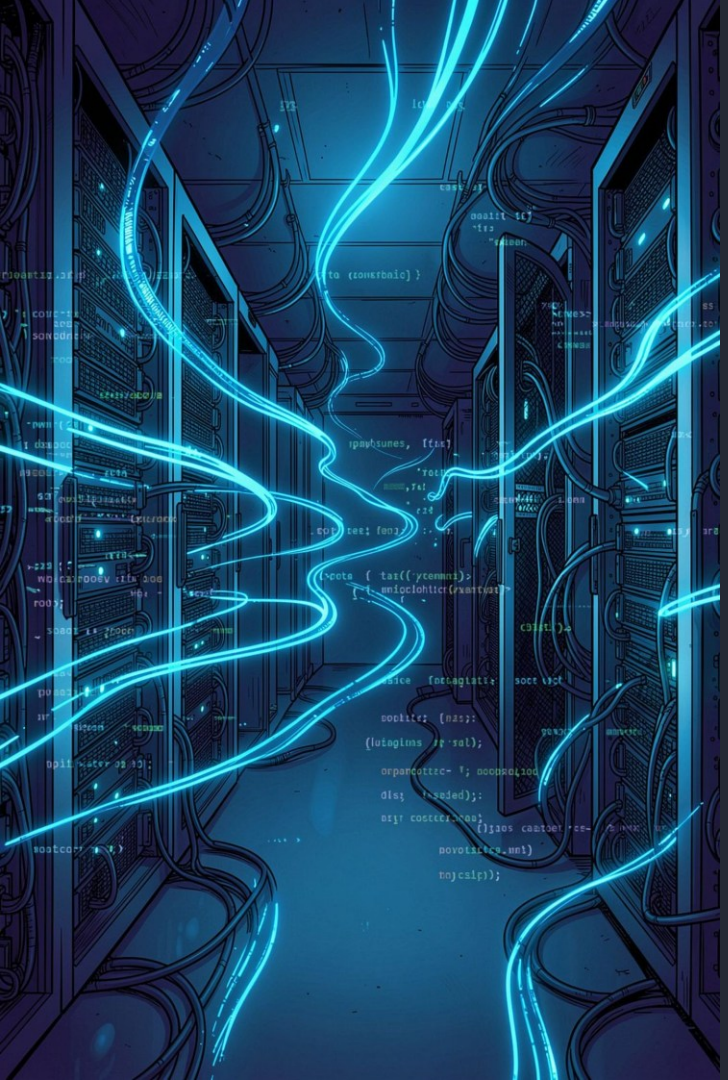


SQL Avanzado — Parte II

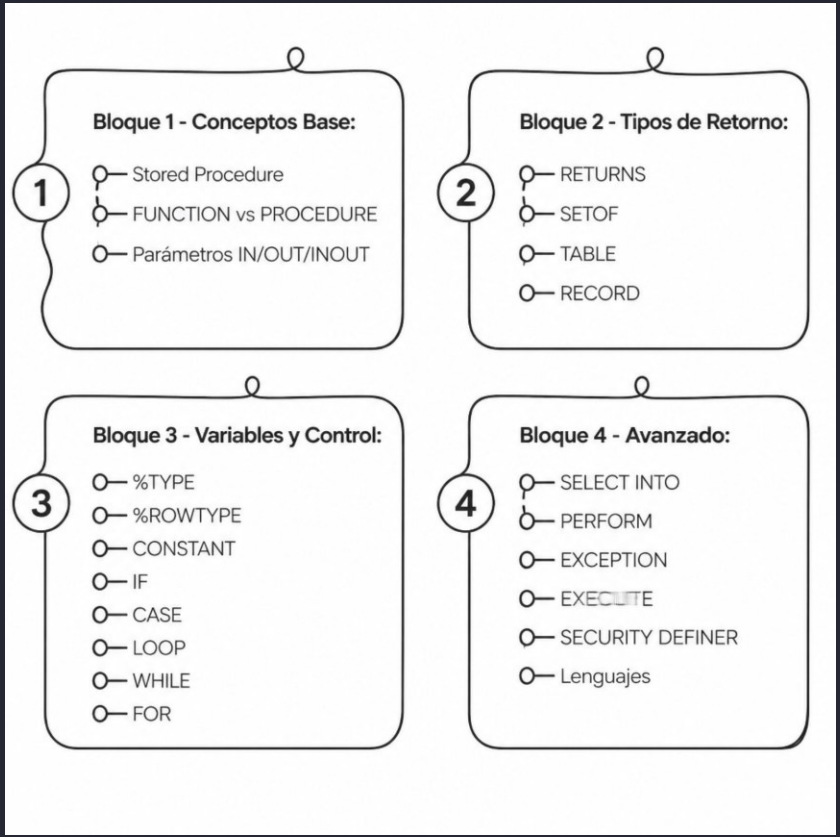
Programación en el Servidor

Funciones, Procedimientos y Control de Flujo en PostgreSQL

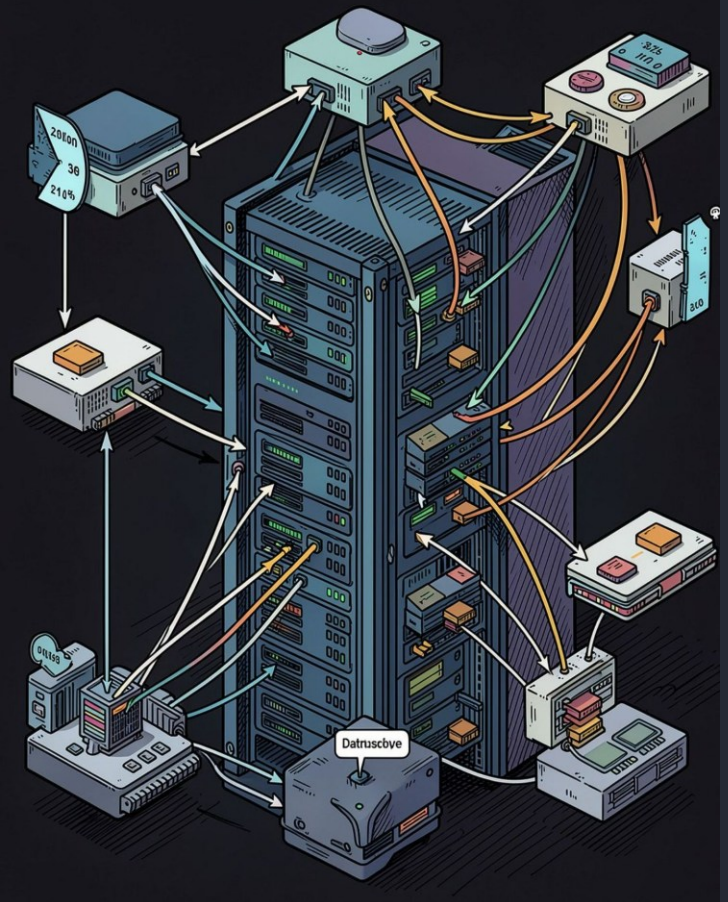
MG. ANDRÉS PASCAL

MG. ADRIÁN PLANAS

Temario de la Clase



Este módulo cubre desde los conceptos fundamentales de la programación en servidor hasta técnicas avanzadas como SQL dinámico, manejo de excepciones y seguridad en PostgreSQL.



¿Qué es un Procedimiento Almacenado?

Subrutina que **reside en el servidor** de base de datos. Centraliza lógica de manipulación: validaciones complejas, procesos batch y generación de reportes.

✓ Ventajas

- Reduce tráfico red-cliente (una sola llamada)
- Reutilización compartida por todas las apps
- Mayor seguridad: permisos sin acceso directo a tablas

⚠ Desventajas

- Lenguaje propietario (dependencia del DBMS)
- Depuración más compleja que en lenguajes de aplicación

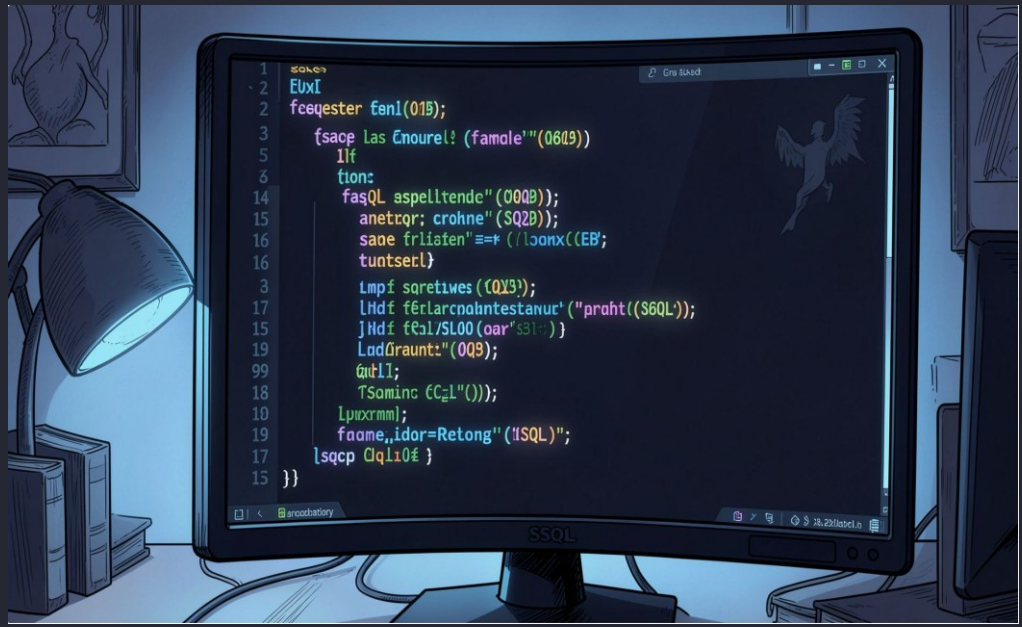
FUNCTION

Sintaxis Base: Función

Las funciones **no manejan transacciones internas**; solo respetan la transacción externa del llamador. Se invocan como parte de una expresión SQL.

```
CREATE [OR REPLACE] FUNCTION nombre(
  param1 tipo, param2 tipo
)
RETURNS tipo_retorno
AS $$
DECLARE
  -- declaraciones locales
BEGIN
  -- lógica
  RETURN valor;
END;
$$ LANGUAGE plpgsql;
```

Invocación: `SELECT * FROM nombre(param1, param2);`



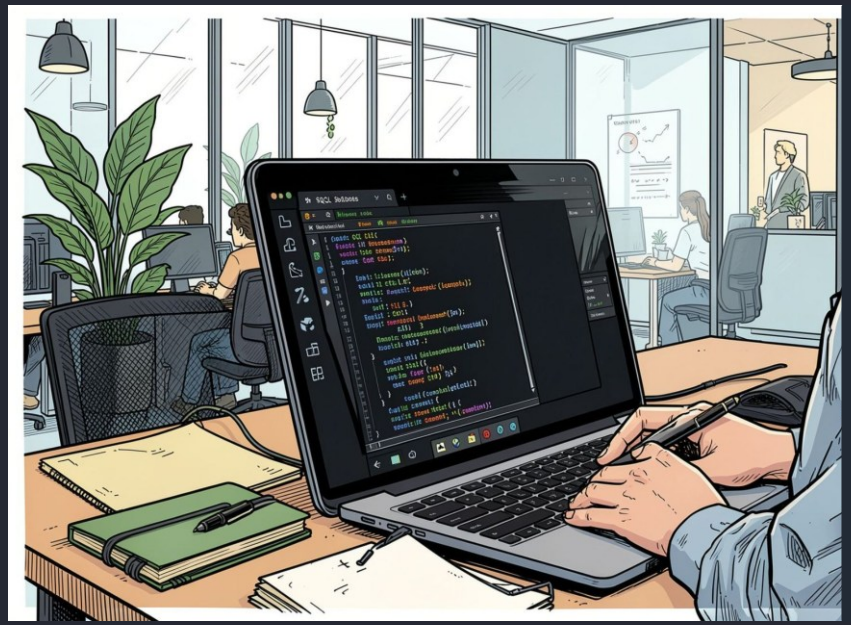
PROCEDURE — POSTGRESQL 11+

Sintaxis Base: Procedimiento

Los procedimientos **sí pueden manejar transacciones** (`COMMIT`, `ROLLBACK`) dentro de su lógica. Se invocan con `CALL`, no con `SELECT`.

```
CREATE [OR REPLACE] PROCEDURE nombre(
  param1 tipo, param2 tipo
)
AS $$
DECLARE
  -- declaraciones locales
BEGIN
  -- lógica con transacciones
  COMMIT;
END;
$$ LANGUAGE plpgsql;
```

Invocación: `CALL nombre(param1, param2);`



Parámetros: Modos IN, OUT e INOUT



IN — Entrada (default)

Solo lectura. Recibe el valor del llamador.



OUT — Salida

Se asigna dentro del cuerpo y se devuelve al llamador.



INOUT — Entrada y Salida

Se recibe, modifica internamente y devuelve.

```
CREATE FUNCTION operacion(
 a IN INT, b IN INT,
 c OUT INT, d OUT INT
) AS $$
BEGIN
 c := a + b; -- suma
 d := a * b; -- producto
END;
$$ LANGUAGE plpgsql;

-- SELECT * FROM operacion(5, 3); → c=8, d=15
```

Tipos de Retorno — Parte I

Tipos Simples

Devuelven un único valor escalar: `INTEGER`, `TEXT`, `BOOLEAN`, `NUMERIC`, etc.

```
CREATE FUNCTION cuadrado(x REAL)
RETURNS REAL AS $$
BEGIN
  RETURN x * x;
END;
$$ LANGUAGE plpgsql;
```

Tipos Compuestos (Registros)

Devuelven una fila de un tipo definido con `CREATE TYPE`. Útil para proyecciones personalizadas.

```
CREATE TYPE empleado_resumen AS (
 id INT,
 nombre TEXT
);

CREATE FUNCTION get_emp()
RETURNS empleado_resumen AS $$
...
$$ LANGUAGE plpgsql;
```

Tipos de Retorno — Parte II

RETURNS SETOF

Devuelve **0 o más filas** de un tipo existente. Puede utilizar `RETURN QUERY` en el cuerpo.

```
CREATE FUNCTION listar_empleados()
RETURNS SETOF empleados AS $$
BEGIN
  RETURN QUERY
    SELECT * FROM empleados;
END;
$$ LANGUAGE plpgsql;
```

RETURNS TABLE

Define la estructura de la tabla devuelta **directamente en la firma**. Más explícito y autocontenido.

```
CREATE FUNCTION filtro(min_sal NUMERIC)RETURNS TABLE(nombre
TEXT, salario NUMERIC) AS
$$
DECLARE
   emp RECORD;
BEGIN
    FOR emp IN (SELECT e.nombre, e.salario
                   FROM empleados e
                   WHERE e.salario > min_sal) LOOP
               nombre := emp.nombre;
               salario := emp.salario;
               RETURN NEXT;
    END LOOP;
END;
$$ LANGUAGE plpgsql;
```

RECORD, Variables y Constantes

RECORD — Retorno Implícito

Tupla **sin estructura fija** en compilación. Útil para funciones polimórficas. Al invocar, se debe especificar la estructura con `AS`.

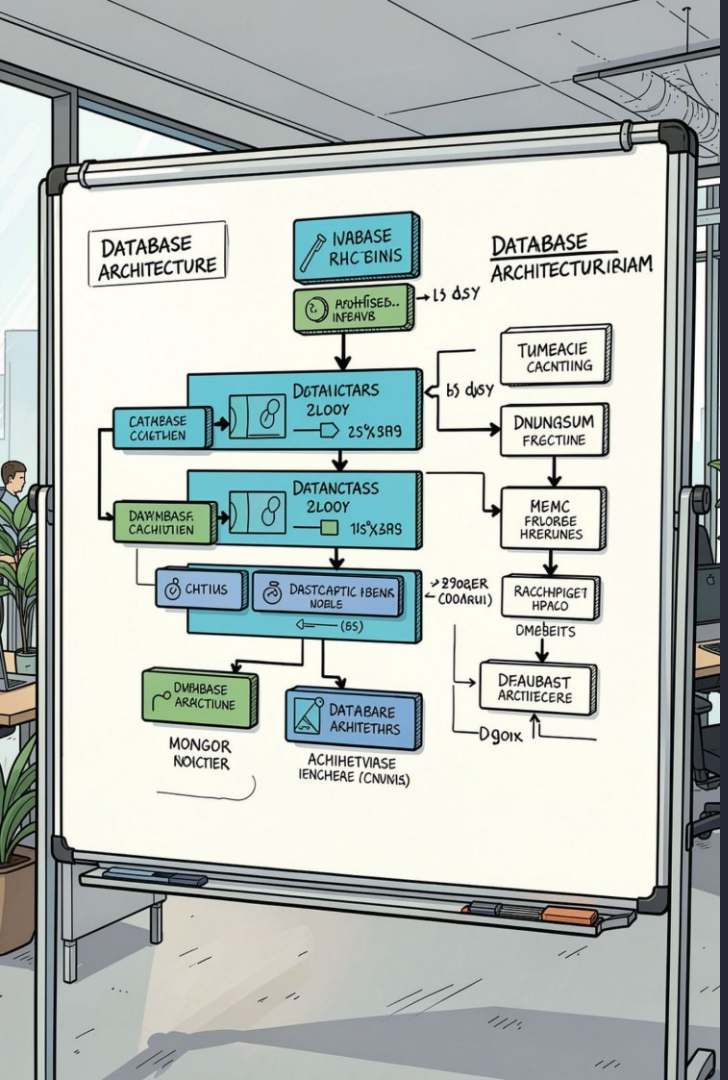
```
CREATE FUNCTION prueba_rec(OUT rec RECORD)
AS $$
...
$$ LANGUAGE plpgsql;

-- SELECT * FROM prueba_rec()
--   AS (id INT, dato TEXT);
```

Declaración de Variables

```
DECLARE
  nombre [CONSTANT] tipo [NOT NULL]
         [:= valor_inicial];
```

- `%TYPE` — Hereda el tipo de una columna: `v_nombre empleados.nombre%TYPE`
- `%ROWTYPE` — Hereda la estructura de una fila: `v_emp empleados%ROWTYPE`
- `CONSTANT` — Valor inmutable: `tasa_iva CONSTANT NUMERIC := 0.21`



Próximos Temas

En la continuación de este módulo se abordarán los siguientes temas avanzados:

01

Estructuras de Control

Condicionales (IF, CASE) y cíclicas (LOOP, WHILE, FOR)

02

Sentencias SQL en SP

SELECT INTO, PERFORM, RETURNING y la variable FOUND

03

Manejo de Excepciones

Bloque EXCEPTION, códigos de error y buenas prácticas

04

SQL Dinámico y Seguridad

EXECUTE, lenguajes PL/pgSQL y PL/Python, SECURITY DEFINER